

# Dreth — Comprehensive Repository Analysis

---

## 1. Project Understanding

---

### What It Is

Dreth is an **experimental simulation framework** for studying how an autonomous agent can build, maintain, revise, and relate learned structure under **limited attention**. It's a research system, not a production ML tool.

### The Core Question

Can an agent keep useful, evidence-shaped structure available while avoiding global trust, global deletion, and flat brute-force rechecking?

### What Problem It Solves

Traditional approaches to causal discovery and structure learning either:

- Trust everything globally (fragile to regime changes)
- Delete/forget aggressively (loses reusable structure)
- Brute-force recheck everything (prohibitively expensive)

Dreth proposes a middle path: **context-indexed, evidence-bounded authority records** over learned structures ("nethras"), with graded trust, cheap sentinel monitoring, and explicit ambiguity tracking.

### The Mental Model

Think of it as an agent that:

1. Observes a hidden causal world through interventions (its only information channel)
2. Discovers structure (parent/function relationships between variables)
3. Earns authority to trust that structure through evidence
4. Monitors cheaply via sentinels instead of re-auditing everything
5. Revokes/demotes when evidence contradicts
6. Tracks ambiguity explicitly (tied hypotheses, dormant alternatives)
7. Records context-dependent roles (a structure can matter in one context and not another)

---

## 2. Technical Architecture

---

### Tech Stack

- **Language:** Pure Python 3 (no framework, no web layer, no database)
- **Dependencies:** Only `random`, `math`, `collections`, `dataclasses`, `json`, `argparse`, standard library. Minimal external deps.
- **No packaging:** No `setup.py`, `pyproject.toml`, or `requirements.txt` exists. Install is manual `git clone` + `run`.
- **Testing:** pytest (8,574 lines of tests across 22 test files)
- **Visualization:** D2 diagrams (`dreth_assessment.d2`), SVG output, matplotlib in scripts

## Code Structure (13,342 lines core + 8,574 tests)

```

dreth/
├── agent.py          (4,186)  # Core package (13,342 lines)
├── ledger.py        (1,140)  # ★ Main control loop  ─ ChainedAgent
└── NethraCertificate, TiedFrontier
    ├── world.py      (854)    # Hidden causal world generator + schedules
    ├── hybrid.py     (772)    # Provider interfaces (advisory only)
    ├── learned_residual.py (685) # Shadow learned-residual experiments
    ├── uncertainty_consolidation.py (605) # Grouping repeated uncertainty
    ├── summary.py     (548)    # Run analysis and rendering
    ├── relative_authority_frontier.py (525) # Graph-frontier proposal priors
    ├── cli.py         (494)    # CLI entrypoint
    ├── shadow_policy.py (482)  # Shadow diagnostic policy layer
    ├── uncertainty_governance.py (444) # Shadow uncertainty extraction
    ├── fit.py         (402)    # Hypothesis enumeration and scoring
    ├── context_role_index.py (365) # ★ Context-indexed nethra provenance
    ├── regime.py      (349)    # Regime detection and cluster sentinels
    ├── relative_authority_observer.py (335) # Post-run graph observer
    ├── shadow_authority_throttle.py (285) # Shadow throttle analysis
    ├── sentinels.py   (190)    # Sentinel selection and validation
    ├── relative_authority.py (176) # Diagnostic relative-authority records
    ├── records.py     (117)    # Diagnostic data structures
    ├── repair_agenda.py (107)  # Pending repair surface
    ├── quality.py     (98)     # Quality-cost scoring
    ├── functions.py   (94)     # Operator vocabulary (9 agent-visible + SIN hidden)
    └── baseline.py    (74)     # Naive refit-everything baseline

scripts/
├── batch_run.py      # Batch runners and summarizers
├── batch_tui.py      # Main batch experiment runner
├── summarize_*.py (9 files) # Terminal UI for batch runs
├── bench_*.py (3 files)   # Various report summarizers
├── visualize.py      # Benchmarking scripts
└── visualize.py      # Visualization

tests/                # 22 test files (8,574 lines)
reports/              # Pre-generated experiment reports (.jsonl, .txt)
docs/                 # Design docs, glossary, ledger notes

```

## Key Data Model

|                                              |                                            |
|----------------------------------------------|--------------------------------------------|
| ChainedAgent                                 |                                            |
| ChainedLedger                                |                                            |
| vars: Dict[int, VarNethra]                   | # One per variable                         |
| parents, func                                | # Current fit                              |
| status                                       | # proposed/certified/uncertain/trass       |
| operation_role                               | # untested/tareth/trass                    |
| certificates: Dict[Op, NethraCertificate]    | # Per-operation authority                  |
| sentinels                                    | # Cheap monitoring probes                  |
| envelope: NoiseEnvelope                      | # Empirical noise model                    |
| compressions                                 | # Memoized predictions                     |
| tied_frontier: TiedFrontier                  | # Ambiguity tracking                       |
| dormant_alternatives                         | # Archived losing candidates               |
| composites: List[CompositeNethra]            | # Joint interaction certs                  |
| hyper_composites: List[HyperCompositeNethra] | # Component-level handles                  |
| novelties: List[NoveltyNethra]               | # Detected novel structure                 |
| CausalWorld                                  | # Hidden oracle (intervention-only access) |
| RegimeRegister                               | # Co-failure pattern tracking              |
| RepairAgenda                                 | # Pending repair prioritization            |
| ContextRoleIndex                             | # Nethra graph provenance                  |
| UncertaintyConsolidation                     | # Uncertainty grouping                     |

## Runtime Loop (Per Cycle)

1. **Observe** current world state
2. **Composite/regime sentinel checks** (cluster-level monitoring)
3. **First-pass dispatch** over variables in topological order:
  - Trass-skip → Compression → Regime-stable → Passive-OK → Sentinel → Queue for audit
4. **Full audit** queued variables (hypothesis enumeration + scoring)
5. **Install** results, run substitution tests, attach sentinels
6. **Record** metrics, context roles, uncertainty signals

## World Simulation

The hidden world is a DAG of variables with causal functions + noise. The agent can only observe through **interventions** (set variable X to value V, observe resulting state). Multiple test schedules stress different failure modes:

- **blind\_challenge** (hardest): 7 relation types including latent oscillators, delayed edges, proxy confounders, phase-switching
- **regime\_switch**: structural regime changes
- **false\_trass**: joint interaction traps

## 3. Current State — What's Implemented & Working



### Solid and Functional

- **Core pipeline:** World → Audit → Fit → Sentinel → Certify → Predict loop is complete and runs
- **Authority model:** Full earned-authority system with operation-scoped certificates, witnesses, revocation
- **Sentinel monitoring:** Cheap-path checks with cost-weighted dispatch, noise envelopes
- **Cascade invalidation:** Graded cascade with route-trass pruning

- **Composite/HyperComposite handles:** Joint interaction detection and component promotion
- **Ambiguity tracking:** TiedFrontier preserves near-tied hypotheses
- **Blind challenge worlds:** Sophisticated stress-testing with 7 relation types
- **Consequence-weight scaling:** Adaptive probe budgets and promotion thresholds
- **Test suite:** 310 passing tests covering core mechanics, cycle behavior, shadow layers
- **Batch experiment infrastructure:** Full CLI, batch runner, multiple summarizers
- **ContextRoleIndex record mode:** Behavior-neutral provenance recording (verified over 60-run sweep)
- **Shadow layers:** Multiple diagnostic-only observation layers (policy, residual, throttle, governance)
- **Extensive documentation:** README, DESIGN\_UNDERSTANDING, TAXONOMY, TODO, glossary

### ⚠ Implemented But Problematic

- **ContextRoleIndex assist\_feature:** Connected and fires, but **over-connects at scale** — increases IV, quality cost, audits, revocations, unique failures, and regime sentinel failures
- **Uncertainty consolidation assist mode:** Can produce harmful broad pressure when clustering is too loose
- **Tied frontier collapse:** Triggers on score-landscape narrowing alone; should require regime-survival evidence (P1.2)

### 🔧 Experimental / Shadow-Only

- Uncertainty governance (shadow diagnostic extraction)
- Shadow policy layer
- Shadow authority throttle
- Learned residual experiments
- Relative authority graph diagnostics
- Temporal frontier evaluation

## 4. Missing Pieces

### Critical Gaps (Blocking Forward Progress)

1. **No project packaging** — No `setup.py`, `pyproject.toml`, or `requirements.txt`. Can't `pip install`. Users must manually ensure Python path.
2. **Context-role match gating** — The #1 documented priority. `assist_feature` over-connects and needs strict matching, deduplication, per-cluster anchor caps, and attribution before it's usable.
3. **18 failing tests** — Mostly in `test_certify_math.py`, `test_fit_transition_boundaries.py`, `test_nareth_divergences.py`, `test_per_operation_certs.py`. All fail with `TypeError: int() argument must be a string, a bytes-like object or a real number, not 'HiddenMutation'` — suggests a recent refactor broke the `perturb_by_schedule` return type handling.

### Structural Gaps

1. **Separating probes (P3)** — `TiedFrontier.separating_probes` field exists but is always empty. Never generated or consumed. This is a planned but unbuilt feature that would help resolve ambiguity.
2. **Dormant alternative revival (P6.2)** — Alternatives are archived but never revived. The data structures track `revival_count` and `context_keys_seen` but nothing reads them.

3. **Compression certificates (Q3)** — `_discover_compressions` doesn't issue a proper `certificates['compress']` cert scoped to the gate predicate.
4. **Regime witness semantics** — Regime handles buy monitoring reduction based on quiescence (absence of failure), not active witnesses. This is a conceptual safety gap.
5. **Authority transaction hygiene** — Some authority objects are still constructed inline in agent code rather than through explicit ledger transactions.

## Developer Experience Gaps

1. **No CI/CD** — No GitHub Actions, no automated test runs
2. **No linting/formatting** — No ruff, black, mypy, or type checking configuration
3. **No API documentation** — No docstrings on most public methods, no generated API docs
4. **No contribution guide**
5. **Stale backup files** — `TODO.md.bak` , `TODO.md.bak2` , `TODO.md.bak3` in repo root

## 5. High-Leverage Low-Hanging Fruit



### Tier 1: Highest Impact, Lowest Effort

#### 1. Fix the 18 Failing Tests (~30 min)

The failures are all the same root cause: `perturb_by_schedule` returns a `HiddenMutation` but callers treat it as an `int`. This is likely a recent refactor that changed the return type. Fixing this would restore test health and unblock development.

#### 2. Add `pyproject.toml` (~15 min)

```
[project]
name = "dreth"
version = "0.1.0"
requires-python = ">=3.9"
dependencies = [] # pure stdlib

[project.scripts]
dreth = "dreth.cli:main"
```

This enables `pip install -e .`, proper imports, and is the minimum packaging for anyone to use the code.

#### 3. Populate `TiedFrontier.separating_probes` (~2 hours)

The field exists. The intervention data is already computed during `fit_var`. The task is to identify which probes discriminate between near-tied candidates and store them. This would immediately improve sentinel selection and frontier resolution — the plumbing is all there.



### Tier 2: Significant Value, Moderate Effort

#### 4. Strict Context-Role Match Gating (~1-2 days)

This is the documented #1 priority. The infrastructure exists (`ContextRoleIndex`, uncertainty consolidation, assist pathways). The task is adding quality filters: shared-var matching, deduplication, per-cluster caps, and attribution counters. The `TODO.md` even has a complete implementation spec (Appendix A).

## 5. Dormant Alternative Revival on Regime Change (~4 hours)

The data structures already track dormant alternatives with revival metadata. Adding revival logic on regime detection would give the agent memory across structural shifts — currently, collapsed alternatives are write-only.

## 6. Cascade Pre-Check Before Re-Audit (~3 hours)

When a sentinel fails and cascades, all descendants get queued for full re-audit. A cheap prediction pre-check could skip descendants that still predict correctly, saving expensive full audits.



## Tier 3: Quality of Life

### 7. Add GitHub Actions CI (~30 min)

Run `pytest tests/ -q` on push. The test suite runs in ~5 seconds. This would catch regressions like the current 18 failures immediately.

### 8. Clean Up Repo Root (~15 min)

Move or gitignore: backup TODOs, PNG files, MHTML zip, standalone Python scripts (`dreth_causal_v28.py`, `drive_inventory_concurrent.py`, etc.) that appear to be exploration artifacts.

### 9. Add Type Hints + mypy (~2-4 hours)

The codebase uses dataclasses but has minimal type annotations on function signatures. Adding types to the core pipeline (`agent.py`, `ledger.py`, `fit.py`) would catch bugs and improve discoverability.

## 6. Unique Advantages & Differentiators

### Conceptual Originality

1. **Context-indexed roles, not global truth** — The tareth/trass model where the same learned structure has different roles in different contexts is genuinely novel. Most systems treat structure as globally true or false. Dreth treats relevance as contextual.
2. **Authority is earned, not assumed** — Every shortcut, skip, and compression must be backed by explicit evidence. This creates a principled alternative to both “trust everything” and “trust nothing.”
3. **Ambiguity as first-class state** — TiedFrontier, dormant alternatives, and the `unresolved / best_available` roles mean the system explicitly represents “I don’t know” rather than hiding it.
4. **Trass is not deletion** — A structure can be operationally irrelevant in one context while remaining useful (and learned) for other contexts. This is a design choice that avoids catastrophic forgetting at the conceptual level.
5. **Shadow/assist separation** — The off/record/shadow/assist ladder is a disciplined way to develop experimental features: prove safety (record == off), then prove usefulness (assist improves metrics). Most research codebases don’t have this rigor.

### Technical Strengths

1. **Pure Python, zero dependencies** — The entire system runs on standard library only. No ML framework lock-in, no deployment complexity. This is rare and valuable for reproducibility.

2. **Comprehensive stress testing** — The `blind_challenge` schedule with 7 relation types (including latent oscillators, delayed edges, proxy confounders) is a sophisticated test bed that goes well beyond typical toy causal discovery benchmarks.
3. **Rich diagnostic infrastructure** — 9 summarizer scripts, JSONL report format, multiple report types, quality-cost metrics. The tooling for understanding what happened in a run is mature.
4. **Well-articulated design risks** — The documentation explicitly lists known failure modes and anti-patterns. This self-awareness is unusual and valuable.

## Potential Leverage Points

1. **The earned-authority model could generalize** — The core idea (explicit evidence-bounded authority over learned structure with context-indexed roles) could apply beyond causal discovery: to any system that maintains learned state under uncertainty with limited compute budget.
2. **Blind challenge as a benchmark** — The `blind_challenge` world generator could be useful as a standalone benchmark for causal discovery methods, independent of the Dreth agent.

## 7. Technical Debt & Issues

### Active Bugs

- **18 failing tests** — `HiddenMutation` return type mismatch in `agent.py` lines 1140 and 3098. Recent refactor broke the contract between `perturb_by_schedule` and its callers.

### Code Quality

- **agent.py is 4,186 lines** — This is the god-object problem. The `ChainedAgent` class handles everything: cycle dispatch, fitting, sentinels, composites, regimes, compressions, context roles, uncertainty consolidation, repair agenda, shadow layers. It should be decomposed.
- **Legacy naming** — `NethraCertificate` (should be “AuthorityRecord”), `nethra_reservoir_mode` alias, `test_nethra_reservoir.py` filename. The conceptual model has evolved past the code naming.
- **Inline authority construction** — Some certs are still built directly in agent code rather than through ledger transaction methods.
- **Stale artifacts in repo root** — Multiple backup files, standalone scripts, PNGs, a zipped MHTML file.

### Architectural Concerns

- **No persistence layer** — Everything is in-memory. A long run’s state can’t be checkpointed or resumed.
- **No parallelism** — Runs are single-threaded. Batch experiments use serial execution (seeds × configs = many runs).
- **Witness replay direction** — Currently runs on pass-path (positive-ledger pattern); should run only on failure (Q5).
- **Compression cert gap** — Compressions don’t issue proper certificates, so their authority isn’t formally tracked.
- **Regime quiescence problem** — Regime handles provide monitoring reduction based on “no recent failure” rather than active validation.

## Documentation Debt

- No docstrings on most methods in agent.py (the critical 4,186-line file)
  - No API reference
  - Internal terminology has evolved but code hasn't been fully updated to match
- 

## 8. Product Perspective

---

### Who Is This For?

Currently: **a single researcher or small research team** exploring earned-authority approaches to structure learning under bounded attention. It's a research instrument, not a product.

### User Value (Current)

- A working simulation of context-indexed authority over learned structure
- Sophisticated stress-testing infrastructure
- A conceptual framework (nethra, tareth/trass, authority records) that's well-articulated
- Reproducible experiments with seed control and batch comparison tools

### Feature Completeness

The core simulation loop is ~80% complete. The missing 20% is:

- Match gating for assist features (the active frontier)
- Frontier resolution (separating probes)
- Dormant alternative revival
- Regime witness maturity

### If This Were a Product

The unique product positioning would be: **"Earned attention allocation for autonomous agents"** — a library/framework that other systems use to decide what to pay attention to, what to trust, and what to re-examine, under bounded compute budgets.

The competitive landscape for this specific niche (context-indexed authority over learned structure) is essentially empty. Causal discovery tools exist (DoWhy, CausalNex, etc.) but they solve a different problem — they find structure. Dreth is about maintaining and using structure under change and limited resources.

### Path to Broader Impact

1. **Fix tests + add packaging** → people can install and run it
  2. **Complete match gating** → the assist pathway becomes usable
  3. **Extract blind\_challenge as standalone benchmark** → attract causal discovery community attention
  4. **Publish the conceptual framework** → the tareth/trass/nethra model is genuinely novel
  5. **Extract the authority model as a library** → other agents can use earned-authority attention allocation
-

## Summary Table

| Dimension               | Assessment                                                                           |
|-------------------------|--------------------------------------------------------------------------------------|
| Core concept            | Strong and original — context-indexed earned authority is genuinely novel            |
| Implementation maturity | ~80% of core pipeline working; experimental layers are early-stage                   |
| Code quality            | Functional but needs decomposition (agent.py god object), naming updates, type hints |
| Test coverage           | Good (310 passing, 22 test files) but 18 tests currently broken                      |
| Documentation           | Excellent conceptual docs; weak API/code docs                                        |
| Packaging/DX            | Missing entirely — no install mechanism, no CI                                       |
| Immediate blocker       | 18 failing tests from HiddenMutation refactor                                        |
| Next strategic move     | Strict context-role match gating (already fully spec'd in TODO.md)                   |
| Unique moat             | The tareth/trass context-indexed role model + earned authority framework             |
| Biggest risk            | Remaining a research artifact without extracting reusable components                 |